

products.csv – Data Exploration Report

Exploration of `products.csv` : structural profile → null/missingness pass → duplicate pass → column-by-column notes across all 34 columns. Full working evidence is in `notebooks/products_exploration.ipynb`.

Structural Profile

- Shape: 728 rows × 34 columns.

Null / Missingness Pass

Column	Null %
s.no	0.0
about_item	0.0
asin	0.0
availability	1.79
best_sellers_rank	23.35
brand_name	0.0
brand_page_url	10.85
breadcrumbs	1.79
customer_review_summary	12.91
default_variant/0	4.4
default_variant/1	9.62
default_variant/2	99.86
delivery_date	3.57
fastest_delivery_date	8.38
list_price	44.92
manufacturer	63.46

Column	Null %
model_number	72.66
price_value	2.88
product_description	62.77
product_url	0.0
rating_count	1.24
rating_distribution/1star-5star	0.0
rating_stars	1.24
recent_purchases	21.84
scrape_time	0.0
seller_name	2.88
seller_page_url	41.48
title	0.0
all_images	0.0
rank_1	25.82

Duplicate Pass

- `asin` duplicates: 0 — fully unique across all 728 rows. Safe key for a `products` collection.

Column-Specific Notes

`availability`

Several issues need unifying:

1. `In Stock` and `In stock` are the same value, differing only by capitalization — needs normalizing in the Transformer.
2. Almost all values will need to collapse to one of 2 options — in stock or unavailable — or roughly 3: in stock, unavailable, and limited.

`best_sellers_rank`

Contains the product's rank in the best sellers list. What matters is the numeric rank itself. For the Transformer:

1. Extract the numbers and provide a new column for the category.
2. Handle the number string → int conversion cleanly.

brand_name

1. 79 brand names contain "Store" in them, needing removal with the pattern `"xxx Store"`.
2. The rest contain the word "Brand" at the start, with the pattern `"Brand: xxx"`.
3. One value is literally the string `"Unknown"`, which should be treated as NaN/null – to handle in the Transformer.

brand_page_url

No suspicious/malformed links found – the anomaly check only surfaced nulls (79 rows), no broken URL structures. Safe to use as-is.

breadcrumbs

1. No need to do heavy parsing on `best_sellers_rank` for category extraction, since `breadcrumbs` already provides the category via a clean separator (the `>` trail).
2. When `breadcrumbs` is null, `best_sellers_rank` is also null – but not the other way around (157 rows have null `best_sellers_rank` with a populated `breadcrumbs`). This rules out using `breadcrumbs` to compensate for missing `best_sellers_rank` values.

default_variant/0, /1, /2

Most of `default_variant/0` holds the size of the variant, with a small amount of pollution (11 rows) containing "color" instead. `default_variant/1` is the mirror case – mostly color, with a small amount of pollution. `default_variant/2` is 99.86% null and not worth transforming further.

delivery_date

1. The dominant value is `"Saturday, March 15"` (610 rows).
2. The rest (92 rows) are scattered across different formats.
3. Main patterns observed: `"day_of_week, month day_of_month"` (most rows), `"month day - day"` (intra-month range), `"month day - month day"` (inter-month range).

fastest_delivery_date

Follows the same pattern shape as `delivery_date`. Worth checking after parsing both columns: whether `fastest_delivery_date` is always earlier than or equal to `delivery_date`, or whether any rows contradict that.

customer_review_summary

Free-text, scraper-generated review summaries — 12.91% null. Populated values are clean and usable as-is; no structural transformation needed beyond deciding how to store free text of this length.

rating_count, rating_distribution/1star-5star, rating_stars

1. This data is **stale relative to the live site** — `rating_count` for one spot-checked product shows `"1,654 ratings"` in the dataset, while the actual current count on Amazon is 2,505, confirmed by manually inspecting the live product page.
2. All rating-related columns are snapshot values tied to `scrape_time`, not live numbers — they will drift from the real site over time. This needs an explicit handling decision in the Transformer: store as a labeled snapshot paired with `scrape_time`, or drop if only current data is wanted.
3. All three column groups are string-embedded numbers (`"1,654 ratings"`, `"4.6 out of 5 stars"`, `"75%"`) needing extraction to numeric types — same shape as `best_sellers_rank`.

list_price

1. Null percentage is fairly high: 44.92%.
2. Once parsed from the `"List Price: $X"` string: max 199.99, min 8.0, mean ~44.73.
3. Keeping nulls as null when transforming is reasonable — no reliable way to compensate a missing list price from other columns.

manufacturer

63.46% null. Populated values are clean — no transformation needed on the values themselves.

model_number

72.66% null; populated values are mostly scattered SKU-style strings with no consistent format worth normalizing further.

price_value

Already a clean `float64` column — min 5.46, max 249.99, mean ~35.32, 2.88% null. No coercion needed, unlike `list_price`, which is still string-formatted.

product_description

Free-text, 62.77% null. Populated values are clean, readable product copy — no structural issues, just a high null rate to account for.

product_url

Zero nulls, zero duplicates — one clean URL per product.

recent_purchases

Follows a consistent "`<N>+ bought`" pattern ("`100+ bought`" , "`1K+ bought`" , etc.) — will need parsing to extract the numeric floor, including handling the `K` suffix as thousands.

21.84% null.

scrape_time

Consistent `MM-DD-YYYY HH:MM` string, all from the same scraping run (March 10, 2025) — no nulls, straightforward to parse to a real datetime. This is the field that should anchor any of the stale rating/pricing values noted above.

seller_name, seller_page_url

`seller_name` is 2.88% null, values are clean (store names or "`Amazon.com`" directly).

`seller_page_url` is 41.48% null — high but expected, since not every seller has a storefront page (e.g. when Amazon itself is the seller).

title

Zero nulls — full-length, descriptive product titles, no structural issues.

all_images

Stores a Python-list-formatted string of image URLs (`"['https://...', 'https://...']"`), not real JSON – will need `ast.literal_eval` or similar in the Transformer rather than `json.loads` , which would fail on single-quoted strings. Zero nulls.

rank_1

Already a clean numeric column (float64) – min 1.0, max 994.0, mean ~174.6, 25.82% null. Appears to be the numeric sub-category rank already extracted from `best_sellers_rank` 's second number (e.g. the `#2` in `"#2 in Men's Jeans"`) – worth confirming this relationship directly in the Transformer, since it may make separately re-parsing that number from the string redundant.

Summary

All 34 columns profiled. Key patterns across the file:

- **String-embedded numbers** needing extraction: `best_sellers_rank` , `rating_count` , `rating_distribution/1star - 5star` , `rating_stars` , `list_price` , `recent_purchases` .
- **Already-clean numeric columns**: `price_value` , `rank_1` .
- **Free-text fields**, high null rate but clean values: `product_description` , `customer_review_summary` , `manufacturer` .
- **Structured-but-inconsistent fields** needing parsing logic: `default_variant/0-2` , `delivery_date` , `fastest_delivery_date` , `all_images` (stringified list, not real JSON).
- **Stale/snapshot data**: all rating-related columns are tied to `scrape_time` and do not reflect current live values – confirmed via manual cross-check against the live product page.
- **Clean, no-issue columns**: `asin` , `product_url` , `title` , `brand_page_url` .

Referential Integrity (against `reviews.csv`)

- Zero orphan `productASIN` values – every review's product reference exists in `products.csv` .
- 700 of 728 products have at least one review; 28 have zero.

- Reviews per product: min 1, median 10, mean ~9.0, max 19. Cardinality is bounded and low — no product has anywhere near a volume of reviews that would threaten MongoDB's 16MB document size limit.